

PUNEETHESWAR POTELA

(+91)9392229814 | potelapuneetheswar@gmail.com | [LinkedIn](#) | [Github](#)

EDUCATION

Indian Institute of Technology Madras

Chennai, Tamil Nadu

B.Tech in Electrical Engineering, Minor in Computing; CGPA: 7.36/10

2021 – 2025

Relevant Coursework

Advanced Computer Architecture(G), Computer Architecture(G), CAD for VLSI(G), Digital Systems Testing(G), Embedded Memory Design(G), Computer Organization, Microprocessors

**G - Graduate level course*

TECHNICAL SKILLS

Programming Languages: C, C++, Python, Bash

Assembly: ARM, RISC-V

HDL: Verilog, SystemVerilog, Bluespec SV

Protocols: SPI, UART, I2C, CAN, AXI4-LITE, JTAG

Debugging/Build Tools: GDB, OpenOCD, Make, CMake, Git

Hardware/EDA Tools: Yosys, nextpnr, Iverilog, GTKwave, Verilator

Hardware Platforms: STM32F103RB Nucleo, Raspberry Pi 3A+, Tang Nano 9k

Systems/Concepts: RTOS, Bare-Metal Development, Memory Mapped IO/peripheral driver development

EXPERIENCE

OLA Electric Technologies

June 2025 – April 2026

Assistant Manager(Electrical and Electronics Team)

Bangalore, Karnataka

Auxiliary Battery for OLA S1 Platform(EV):

- Engineered low-voltage **AUX battery module** to retain handle, seat lock access during **main-pack power loss**
- Optimized cell configurations and circuit behavior for **dynamic current loads** while adhering to thermal limits
- Maintained **supply chain consistency** by repurposing existing **main-pack cells** for the AUX battery design

OLA Roadster and Diamondhead Platforms(EV):

- Developed the **Electrical Systems Architecture** for the **OLA Roadster** and **Diamondhead EV platforms**
- Designed **signal routing**, **power distribution**, and **pinouts** for core ECUs like **BMS**, **MCU**, **LCM**, **VCU**
- Standardized the front and rear **junction boxes** via **common flat cable architecture** across EV variants
- Executed critical hardware bring-up, and validation to deliver the **Diamondhead announcement prototype**

OLA Shakti Platform(BESS):

- Spearheaded **electrical architecture**, defining **power topologies**, **isolation barriers**, and **components**
- Built and validated the compliance-ready **homologation prototype** to meet the regulatory safety standards
- Defined **End-of-Line(EOL)** test specifications, pass/fail thresholds for high-voltage and thermal safety limits
- Trained an **8-member team** at OLA Futurefactory in **assembling procedures** and **EOL test requirements**
- Resolved the **cross-functional integration dependencies** by bridging the **hardware** and the **firmware teams**

Mindgrove Technologies

May 2024 – July 2024

RTL Design Intern

Chennai, Tamil Nadu

Indirect Branch Predictor (RISC-V Shakti C-Class):

- Analyzed pipeline architecture to implement **dynamic branch target buffering** for optimized instruction fetch
- Engineered a **target address buffer** with a **two-stage cascaded predictor** to balance latency and accuracy
- Enhanced decode logic to seamlessly support **32-bit Base** and **16-bit RISC-V Compressed** instruction sets
- Integrated the validated predictor module into the production-grade **Shakti C-Class core pipeline**
- Validated performance via **CoreMark benchmarks**, achieving **99% accuracy** with zero pipeline regressions

Custom Preemptive RTOS | *C, ARM Assembly, Embedded Systems, Bare-Metal, GDB* | [Details](#)

RTOS Kernel:

- Developed a bare-metal, preemptive **Round-Robin** scheduler utilizing SYSTICK-based preemption to handle multi-tasking execution workflows on an **ARM Cortex-M3 (STM32F103RB)** microcontroller
- Designed and implemented a strict **Kernel-Port contract** to explicitly decouple core kernel mechanics from architecture specific port layers, enabling highly modular and **platform-independent** software execution
- Developed deterministic synchronization primitives, including blocking **Semaphores** and **Mutexes**, to systematically guarantee thread-safe shared resource contention and reliably prevent critical data races
- Designed an explicit **task lifecycle state tracking** system with support for **task deletion** and **self deletion**
- Designed kernel instrumentation hook to analyze real-time timing performance metrics via **GDB scripts**
- Profiled kernel performance on ARM to demonstrate a deterministic, constant $\mathcal{O}(1)$ **74-cycle context switch latency** with a predictable tick overhead of $\mathcal{O}(n)$ at **65 cycles per task**

Bare-metal UART driver:

- Programmed a custom bare-metal, **interrupt-driven** UART driver utilizing **ring-buffered circular queues** to successfully enable continuous, non-blocking, and asynchronous serial communication
- Designed an interactive, lightweight **UART-based CLI Shell** acting as the user-space interface to **demonstrate kernel features** and active system states

Custom RISC-V SOC | *Verilog, RISC-V ISA, AXI4-Lite, FPGA, Spike Simulator* | [Details](#)

Core and Modules:

- Designed and implemented a synthesizable **RV32I ISA** based RISC-V processor core incorporating the **Zicsr** extension and supporting **Machine-mode** external and timer interrupts.
- Engineered an efficient, **non-vectorized** interrupt mechanism that routes all exceptions and hardware interrupts to a centralized, **common hardware trap handler**
- Developed an **automated verification framework** that cross-checks core execution traces against the **golden Spike ISA simulator** to ensure 100% architectural compliance
- Integrated the processor core, memory controllers, and memory-mapped peripherals over an **AXI4-Lite** bus interconnect protocol to ensure modular System-on-Chip (SoC) expansion
- Designed a **hardware timer peripheral** fully compliant with the RISC-V Privileged Specification, exposing standard **mtime** and **mcycle** registers to the software layer
- Developed a custom GPIO peripheral module, mapped it to the AXI4-Lite bus, and successfully targeted, routed, and validated the design on a **Tang Nano 9K FPGA** utilizing physical onboard LEDs

HW-SW Co-Verification | *Hardware-Software Co-Design, SoC Porting, System Simulation* | [Details](#)

RTOS and RISC-V SoC Integration & Profiling:

- Ported the above custom **RTOS kernel** to the above custom **RISC-V SOC**, aligning software context-saving mechanism with the hardware's minimal **common trap handler** philosophy
- Engineered custom simulation testbenches optimized for printing granular **execution traces** and exporting VCD waveforms, drastically reducing debug cycles during **hardware-software co-debugging**
- Designed a specialized system-level **simulation harness** to measure the exact cycle-accurate **hardware/software timing interactions** of the preemptive kernel running on top of the RISC-V silicon
- Characterized the RISC-V port's trap overhead via cycle-accurate benchmarking, measuring a minimum trap overhead of **987 cycles** and an average execution cost of **1,013 cycles**